

BESK TECH · AI SYSTEMS

Agentic Text-to-SQL, *in production*

VLADIMIR BESKOROVAINYI · CTO / CPO

ARXIV:2606.29733 · ORCID 0009-0004-7005-6242



Two sources in this *talk*

Production experience

You cannot verify it

Internal measurement, closed dataset,
our own definition of correctness.

Public benchmark

You can regenerate every number

BIRD dev, released predictions,
reproducible end to end.

Not comparable. I will say which kind every number is.

A demo and a production system are not the same *problem*

	DEMO	PRODUCTION
Schema	5 tables	500+ marts
Who asks	You	30 million people
Cost of an error	A bad screenshot	A decision on a wrong number
Infrastructure	A laptop	On-prem GPUs, no egress
Who answers for it	Nobody	Me

Production: internal measurement, closed dataset. Benchmark: BIRD, public, reproducible.

BIRD dev · n = 1534 · Execution Accuracy

Qwen2.5-Coder

7B / 14B / 32B

CodeLlama-Instruct

7B / 13B / 34B

Llama-3.x

8B / 70B

Greedy decoding · vLLM · on-prem fp16 · identical prompts across every model

Paired exact McNemar test. A naive confidence interval at n = 1534 is 2.5 pp.

18 pp

separates two families at the same 7B size

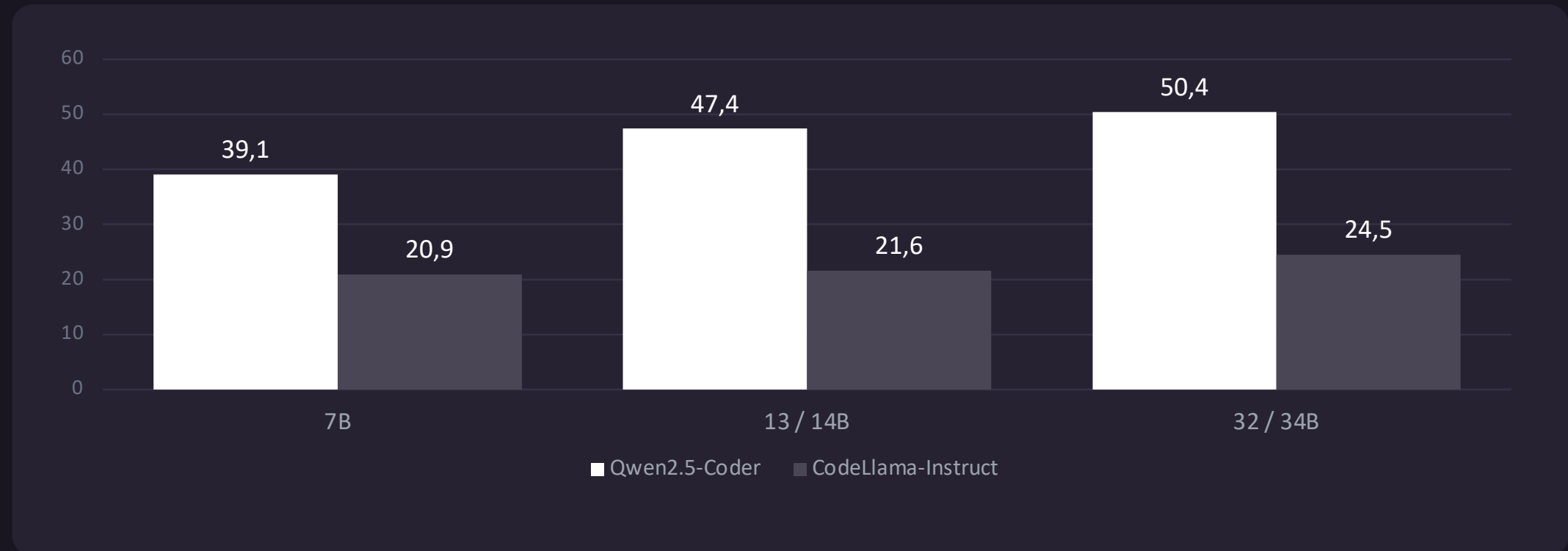
39.1 → 50.4

Qwen2.5-Coder

20.9 → 24.5

CodeLlama

Generation beats *size*



Llama-3.3-70B 49.2 vs Qwen-32B 37.6 on a quantised endpoint, relative comparison only.

Self-correction is cheap, and it *generalises*

MODEL	DELTA	P
CodeLlama-7B	+3.06 pp	2e-11
Llama-3.1-8B	+3.65 pp	6e-11
CodeLlama-34B	+2.61 pp	4.2e-9
Qwen-32B	+1.24 pp	0.0013
Llama-3.3-70B	+1.04 pp	0.08 n.s.

It never hurt in any run.

What it actually *fixes*

FIRST ATTEMPT

```
SELECT m.name, avg(a.amount)
FROM appeals a
JOIN marts m ON m.id = a.mart
GROUP BY m.name
```

executes, wrong join key

AFTER SELF-CORRECTION

```
SELECT m.name, avg(a.amount)
FROM appeals a
| JOIN marts m ON m.id = a.mart_id
| WHERE a.status = 'closed'
GROUP BY m.name
```

correct key, filter restored

One extra pass, same model, no fine-tuning.

96.5%

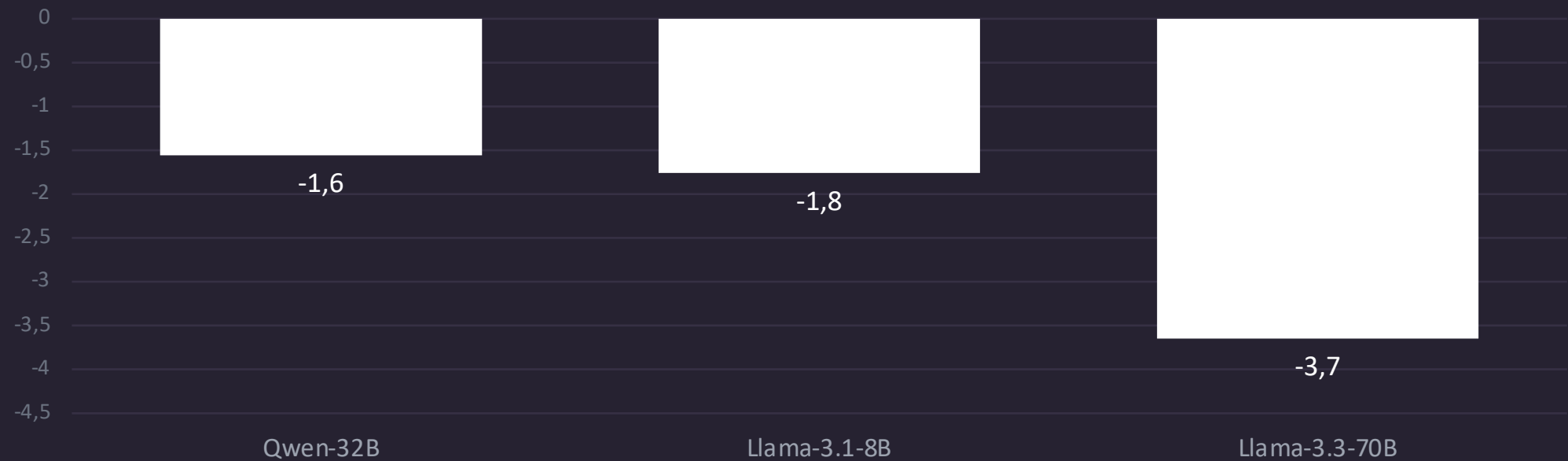
gold-table recall from the embedding linker

and still

no gain

on any model

Schema linking does not *help*



$p = 0.035$

$p = 0.03$

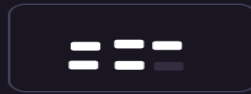
$p = 1.4e-6$

The linker retrieves the right tables. Accuracy still goes down.

Retrieval means two different *things*

Same name, two different jobs

5 tables



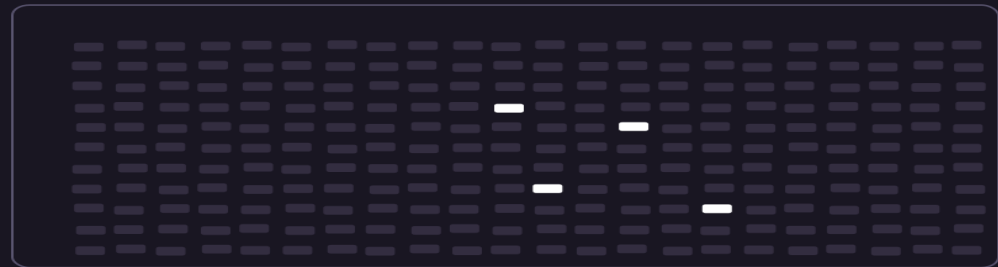
Retrieval is an optimisation

Optional, and measurable

It measures negative

Skip it

500+ marts



Retrieval is a precondition

The schema does not fit

Routing makes it possible

Required

The model may be wrong. Not *silently*

01

Blocked unconditionally

Read-only access
Objects by whitelist
Row limit on every query

02

What survives all of that

Valid SQL
Wrong business logic
Nothing to block

03

Why the review stays

A person sees the number
before it becomes
a decision

The model is allowed to be wrong. It is not allowed to be wrong silently.

+0.13 pp

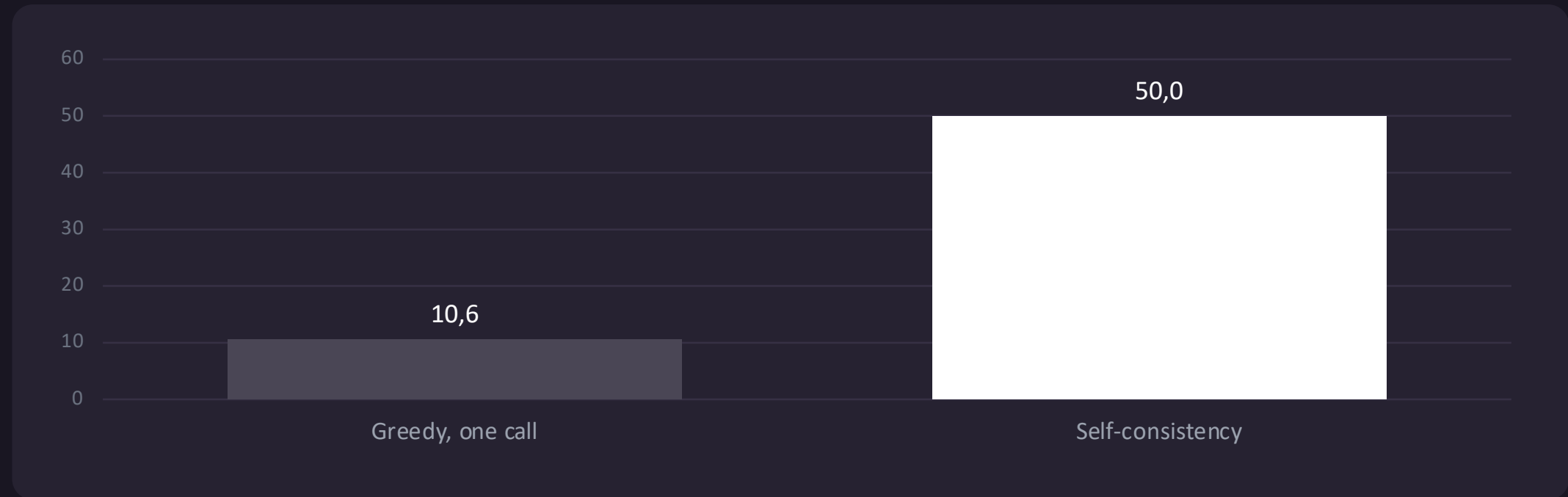
for five times the tokens

$p = 0.86$
not significant

10.6 s $\rightarrow$ 50.0 s
p95 latency

COST

Self-consistency is not worth *it*



Self-correction, for comparison: \$0.12 to \$0.22 per 1k queries.

Cost from wall-clock at \$1.8 per GPU-hour, not a token proxy.

LIMITATIONS

What this study does not *show*

THIS STUDY, BEST ON-PREM

50.4%

CLOUD SINGLE MODELS

~57%

RL-ENHANCED SYSTEMS

71.8%

Zero-shot · one greedy call · no fine-tuning

BIRD has documented annotation errors

Part of the results ran on a quantised endpoint

Guardrails were not measured

Four things to take *away*

- 01 **Recent family before big model**
- 02 **Add self-correction**
- 03 **Measure linking before you trust it**
- 04 **Test differences, not deltas**

Everything here is *reproducible*

Preprint	arXiv:2606.29733
Code	github.com/beskvladimir-create/nl2sql-onprem-bench
Data and DOI	Zenodo 10.5281/zenodo.20952794 · MIT

Two things I would like someone to do

Run it on a wide-schema database

Add a same-generation code-specialised family

BESK.TECH · [LINKEDIN.COM/IN/VLADIMIRBESK](https://www.linkedin.com/in/vladimirbesk)